From Mined Fixes to Gated Repair: Closed-Loop Defect Synthesis for CIRCT

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla
International Institute of Information Technology, Hyderabad
{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

***Abstract*—A bug in a hardware compiler such as CIRCT can put a silent defect into a chip. Repair agents can fix the bugs people report, but people report few, and the open ones grow old. We built a closed loop on the CHIA agent framework that manufactures its own bug supply: a model reads each of 187 past CIRCT fixes, learns the mistake behind it, and writes new inputs that expose the same mistake elsewhere. Deterministic tools build, judge, reduce and deduplicate every input, CHIA's existing repair chain drafts the patch, and a named person approves every filing. We also restored the compiler's own assertions, which CHIA's published image had compiled out, in 536 of its 555 objects, so the checks its authors wrote can fire again. In two campaigns the loop found fifteen new bugs, each reproduced, minimised, validated and unmatched against 2,652 issues, while a mutation baseline over 2,308 inputs found none. Both campaigns together read 205 million tokens, three of every five from cache, and wrote 3.7 million. A maintainer receives a bug already reduced, checked and paired with a candidate fix.**
Source: github.com/DevAbhinav-23/CHIA_Hackathon_2k26

***Index Terms*—Hardware compilers, Program repair, Test generation, Compiler fuzzing, CIRCT.**

I. Introduction

CIRCT is an open-source compiler for hardware, built on MLIR [1]. A designer describes a circuit in FIRRTL or SystemVerilog, CIRCT lowers it to Verilog, and synthesis tools turn the Verilog into a chip. When CIRCT crashes, the designer sees the crash and works around it. When CIRCT emits wrong Verilog, nobody sees anything: the output passes every tool below the compiler and can reach fabrication. CIRCT tests the LLVM way. Each regression test carries the command to run and the output to expect, and a tool called `lit` runs them.

CHIA [2] is the agent framework this hackathon builds on. Its CIRCT issue solver takes a GitHub issue through six phases: it assesses the issue, reproduces it, fixes it, runs the whole test suite, repairs regressions, and writes the pull request. On 16 CIRCT issues it fixed five, and maintainers merged three. Its bottleneck is supply. Over the past year CIRCT received 82 bug-labelled issues and closed 65. The backlog does not drain: 101 issues stay open, at a median age of 911 days (Fig. 1a). The bug a maintainer wants is small, already checked against known issues, and comes with a candidate fix, from a source that files nothing else. A past fix that corrects one wrong assumption inside a pass usually leaves the same assumption at other sites. CIRCT ships no fuzzer to find them, and a naive loop would be blind to most of them (Fig. 1b).

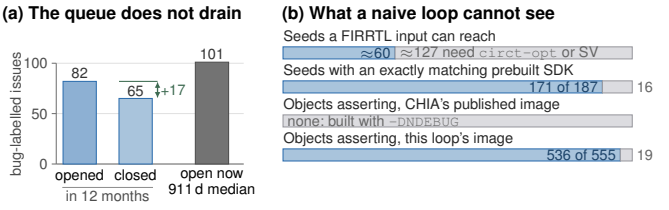


Fig. 1. Motivation. (a) CIRCT closes fewer bug issues than it receives, and what stays open is old. (b) What a mined-fix loop must overcome.

The main idea of this paper is simple: an agent reads each past fix and writes inputs aimed at the sibling sites, and tools then discard almost everything it writes. We make four claims.

- We build a loop in which the model acts three times, reading a fix, writing inputs and writing the report, and a tool answers every gate (Section II, Fig. 2).
- We rebuild 171 of 187 historical CIRCT commits in minutes from prebuilt SDKs, and we restore the assertions CHIA's image compiled out, in 536 of 555 objects, without changing which of CIRCT's tests pass (Section III, Table I).
- We pre-register every threshold by git tag, run a mutation baseline on the same harness, and measure the dedup rule on 22 labelled pairs (Section IV).
- We put fifteen new CIRCT bugs through a four-question gate in two campaigns, the baseline in those campaigns puts none, and we read every number from the loop's database (Section V, Table II).

II. The Loop

Fig. 2 names the stages and marks the ones the agent runs. Tools decide everything else. One model serves every agent stage, Gemini 3.8 Flash through CHIA's Vertex backend, and no stage can build a client for it unless the operator sets one environment variable for that run. A test or a dry run therefore cannot reach the model.

Read the seed (1). A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, together with its diff and the test its author added or changed. The agent reads both, states in one sentence which root-cause class the fix corrected, and names other sites where the same assumption still stands.

Write probing inputs (2). The seed's own test decides the language, because its command line names the tool through which the bug was reachable: `circt-opt`, `firtool` or the

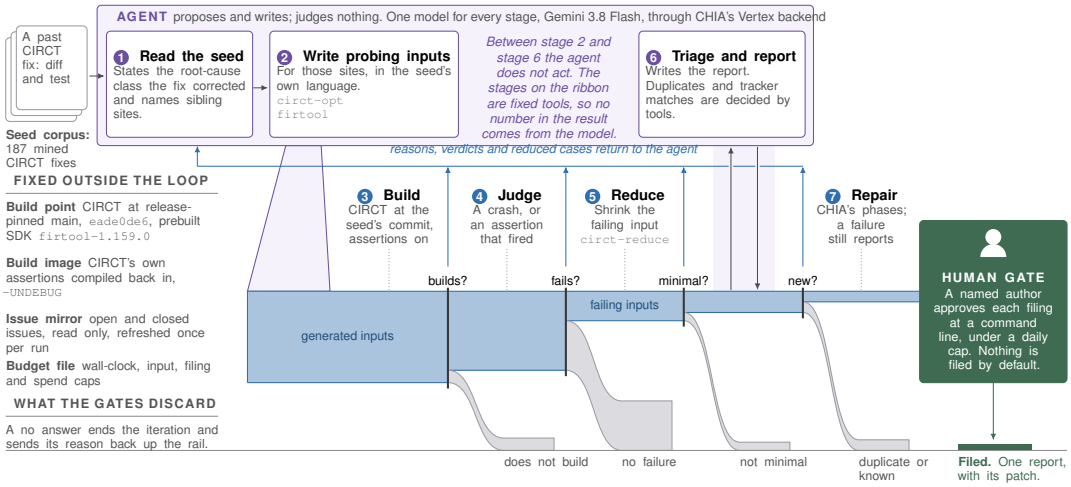


Fig. 2. Seeded CIRCT bug loop. The agent holds the violet band and acts three times only, at stages 1, 2 and 6. The blue ribbon below it is fixed apparatus, so no number in the result comes from the model. The ribbon is the candidates in flight, pinched at each gate by a yes-or-no question, and the share answering no peels onto the floor. Widths are illustrative.

SystemVerilog front end. About 127 of the 187 seeds sit in code that no FIRRTL input can reach, so a FIRRTL-only generator would miss most of the corpus. The agent reads the compiler's source and writes each candidate through tools of its own. It runs no command.

Build (3). CIRCT compiles against one pinned commit of LLVM and MLIR, and Section III explains how the loop avoids building those from source. The build compiles in CIRCT's assertions, the internal checks its own authors wrote, which stop the compiler the moment an assumption breaks.

Judge (4). An oracle is the rule that decides whether an input found a bug. The primary oracle admits no argument: the compiler crashed, or an assertion fired. A second oracle compares CIRCT's simulator with Verilator and only reports, because the two disagree routinely over uninitialised and reset signals.

Reduce (5). A crashing input is usually large. `circt-reduce`, which CIRCT ships, deletes parts of it and keeps each deletion while the failure survives. A textual reducer takes over where that tool dies.

Triage and report (6). Many failures are one failure, and deciding which are the same is mechanical. A fingerprint is the assertion text with its source location or, for a crash, the symbolised frames that remain after the loop strips the crash handler's prologue. It matches survivors against a mirror of CIRCT's issues and against commits made since the build point. The agent writes the report, and no gate reads what it concluded.

Repair (7). CHIA's phase chain runs unaltered except for one branch that selects our backend and an entry point that takes a local report in place of a GitHub issue. We restrict repair to crashes and assertion failures, where the solution is unambiguous.

The gates. Between stages sit questions with two answers, the bars that pinch the ribbon in Fig. 2. A no ends the iteration and returns its reason to the agent with the verdicts and reduced cases. The last gate asks four questions, and a tool answers each: does the failure reproduce in a fresh process on another worker, is the case minimal, is the input valid, and is it new. The third question runs three parse-and-verify commands, and a clean rejection makes the input the loop's fault rather than the compiler's. After the tools, the gate is a person: a named author approves every filing at a command line, and the loop files nothing by default.

TABLE I
WHAT WE MEASURED ON THE IMAGE THE LOOP RUNS

Measurement	Result
Building the past cheaply	
Fix-shaped commits mined from 24 months	187
Of those, seeds with an exactly matching prebuilt SDK	171
Pin windows with no matching release, worst 16 days	16 of 63
Letting the compiler complain	
Compile commands carrying `−UNDEBUG`	778 of 778
Object files carrying the assertion handler again	536 of 555
CHIA's whole test gate, assertions on and then off	1,119 pass, 0 fail, identical
What the assertions cost: wall clock, bytes, per probe	18%, 25%, 4%
Tests `lit` discovers, once its source root is the SDK	1,390, 0 errors

III. Measured Preconditions

The loop has to build CIRCT at arbitrary points of its history, and the compiler has to be able to complain. Neither held when we started. Table I lists what we measured on the image the loop now runs.

Building the past cheaply. CIRCT records the exact LLVM commit it expects, its pin. A build against any other commit fails, and compiling that LLVM costs hours per seed. CIRCT also publishes an SDK of prebuilt LLVM and MLIR libraries that carries its own pin, and where the two pins match, CIRCT rebuilds in minutes. 171 of the 187 seeds do. The same mechanism fixes where the loop runs: at release-pinned main, the newest commit whose pin has a published release, 8 commits and 4.8 days behind the head for this image. A long lag lowers the filing rate, because more candidates come back already fixed.

Letting the compiler complain. Optimised builds delete assertions by defining the macro `NDEBUG`, and CHIA's pub-

lished CIRCT image is such a build. Its binaries hold no reference to the assertion handler, so a loop whose oracle is a failed assertion would find nothing under it. One compiler flag, `-UNDEBUG`, puts the assertions back. The image the loop runs is that build at release-pinned main, with the flag in every compile command and the handler back in all but 19 of its 555 compiler objects, which assert nothing.

The oracle rests on whether the restored checks fire on CIRCT's own tests. We therefore built the image twice, differing in that flag alone, and ran CHIA's whole test gate in both. The two runs pass the same 1,119 tests and fail none, so a probe that trips an assertion trips it because of what the probe does. We had to repair one defect first: at CIRCT commits after mid-2026 a test directory's configuration raises while `lit` discovers tests, and CHIA's exclusion of that directory hid the error, so the gate discovered nothing. Pointing the build at the SDK as its MLIR source root restores discovery.

Tested on real failures. We calibrated the oracle and its fingerprints on six past CIRCT bugs, rebuilt at the commit before their fix and recorded with input, argument vector and error output. Two defects in the fingerprinting rules appeared only there: the name normaliser erased every frame in an anonymous namespace, where CIRCT's passes live, and the scope test read an inlined frame instead of the physical one. Together they put three of the four real bugs outside the repair stage's reach. We fixed both.

IV. Controls

The loop embodies one experiment: does reading a past fix make an agent better at finding new bugs than mutating the same tests at random? Two arms share the harness, oracles, reducer and gate; only the generator differs, and with it the information it sees. The mutation arm rewrites seed tests syntactically, on 96 of the 184 eligible seeds, without seeing the diff or the root-cause class, following Mut4All's design for mutators that a language model synthesises [3]. Both arms receive the same budget, a fixed window of wall-clock seconds, but the mutation arm exhausts its mutants after 775 and 266 seconds, so only the seeded arm uses its window; a post-review run that used it gated 2 bugs from 115,869 inputs. The registered headline is distinct bugs a CIRCT maintainer confirmed by labelling, commenting on or fixing them; until filing, we report bugs through the gate.

Pre-registration. Every threshold lives in one budget file, and an annotated git tag on the commit that lands that file registers the run. The loop refuses a file no tag names and a mutator set committed after the tag, so nobody can choose a threshold after the fact. Safety caps bound damage rather than effort: 2,000 inputs per arm per day, 3 filings per day and 10 per run, 60 seconds and 4 GiB per probe, and a hard cap on model spending. The loop files nothing until a human has posted the method to the project's forum.

Deduplication. Two candidates are one bug when their primary fingerprints are equal. On a hand-labelled set of 22 candidate pairs the rule merges 1 of the 11 distinct pairs and splits 1 of the 11 duplicate pairs, and we print both rates beside the headline.

TABLE II
Both campaigns, from the loop's database. A seeded column is its two shards together. The mutation column is one run per campaign, and the two runs agree in every cell

Measure	seeded 1	seeded 2	mutation
Seeds probed	9	43	96
Probes written	134	624	1,154
Output rejected by its own verifier	10	7	0
Assertion failures	5	54	0
Crashes	0	8	0
Candidates	15	69	0
Distinct new fingerprints	3	14	0
Through the gate	**3**	**12**	**0**
Filed	0	0	0
Model turns	57	191	0
Input tokens, millions (share from cache)	56.0 (68%)	148.7 (59%)	0
Output tokens, millions	0.72	3.00	0

Known issues. A mirror of the tracker answers whether a candidate is already reported. GitHub stops paginating at its hundredth page, so the mirror walks newest first, then oldest first, and unions the two: 2,652 issues in 198 requests.

Contamination. CIRCT's fixes could be in the training data of any model we can use, so the agent may reproduce a fix it memorised. We screen every candidate against fixes made after its seed, at the level of the symbols the seed's fix touched. That screen flags 77% of the corpus where the same screen at file level flags 96%. A real crash stays a real crash, so the flag marks the candidate and does not exclude it.

V. Results

Table II reports two registered campaigns, each running the seeded arm as two shards in parallel. The first gave every arm two hours under a fixed spending cap per job. We started it three times and stopped the first two early for defects the live run exposed. The second changed three registered numbers and nothing else: the whole corpus instead of its head, a 41.7-hour window and a higher cap. We stopped it ourselves after 11.9 hours, well inside both. We read every number from the loop's database after re-judging the stored probes under the corrections below, and we re-ran no model turn.

The yield is stable, and the baseline trails far behind. Fig. 3 reads Table II as rates. The seeded campaigns differ fivefold in size, yet they yield 11.2 and 11.1 candidates and 2.2 and 1.9 gated bugs per 100 inputs. The mutation arm wrote three times as many inputs as both seeded campaigns together, and none fired the oracle; only 34% of them reached the pass they target. A gated bug cost 18.7 million input tokens in the first campaign and 12.4 million in the second. Fifteen bugs show a consistent rate. They do not measure an effect size.

Fifteen new bugs through the gate. The first campaign's three bugs all come from the August `moore.extract` seed: the zero-element array trips the assertion at `HWOps.cpp:1995`, and a dynamic extract and an extract of a reference lower to `comb.extract` and `llhd.sig.extract` with aggregate types. The second campaign's twelve spread across the compiler, and three of them are not assertions at all:

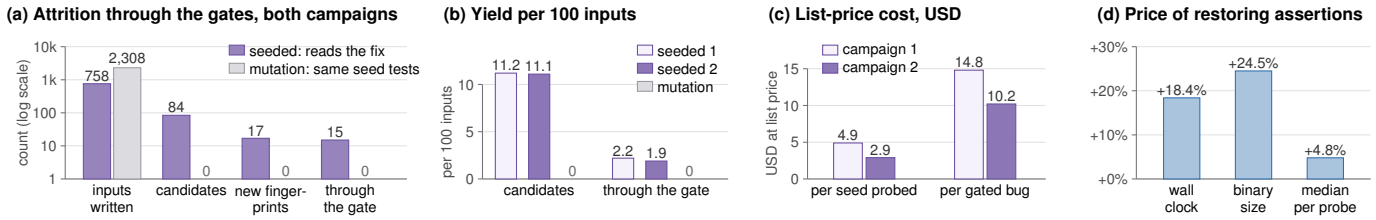


Fig. 3. Both registered campaigns (Table II). (a) One harness, oracle and gate; only the generator and its information differ. Zeros sit at the log axis floor. (b) Rates agree though campaign 2 is five times larger. (c) Model use per gated bug at list price, USD 0.75 and 3.75 per million input and output tokens. (d) Overheads of Table I.

a segmentation fault in `resolveClassStructBody` in `MooreToCore.cpp`, a second whose only named frame is `main` in `circt-opt.cpp`, and an abort in `ElaboratedObjectOp::verifySymbolUses` in `OMOps.cpp`. Each of the fifteen reproduced on another worker, was minimal at the reducer's fixpoint, was valid, and matched neither an issue nor a commit since the pin. A named human holds the reports the model wrote. Two further new fingerprints did not survive the gate: one did not reproduce on re-run, and the validity checker rejected the other.

The oracle had to learn a class. Seventeen of the seeded arm's rejected probes were the compiler's fault and not the model's: the input alone parses and verifies, and the diagnostic has the shape of MLIR's generated verifier. We require both conditions, since a pass may complain legitimately about a valid input, such as two top-level modules. No rejection in either mutation run qualified, because a mutated test that fails to parse is broken syntax. Across 2,308 mutants that arm raised no candidate to judge.

Dedup needed four corrections, and we found each on a real candidate. A post-pin commit counted as a fix if it touched the crash's file, so an LLVM bump touching 87 files silenced the first find. It now has to touch a frame symbol. A generic MLIR assertion text matched an open issue, and now also needs a CIRCT frame token. A verifier failure matched an issue on the op name alone, and now needs the constraint. Two candidates sharing a fingerprint were each the other's duplicate, which left thirteen without a representative. The loop recomputes every verdict from stored evidence, so a corrected rule re-judges every candidate without a model turn.

Tokens. The first campaign read 56.0 million input tokens over 9 seeds and the second 148.7 million over 43, about 6.2 and 3.5 million a seed, and they wrote 0.7 and 3.0 million output tokens. Almost all of the reading is re-reading. CHIA's backend resends the whole conversation on every tool call, so a turn with twelve calls reads its context twelve times over, and the cache served 68% and 59% of that input.

Repair. We made three attempts on the assertion. The first two never reached a patch, because the loop's own script handed the tool two input files. With that fixed, the chain reproduced the failure, wrote a one-line change, rebuilt CIRCT, ran its test gate, and hit the 45-minute wall while writing up. The change was a debug print. We claim no fix.

VI. Related Work

SWE-bench [4] scores agents on GitHub issues, and CHIA's issue solver [2] applies the pattern to a hardware compiler. Both draw from a queue, and we fill one. Fuzzers fill queues without repairing or withholding: CSmith [5] for C, and for MLIR, MLIRod [6], Nüwa [7], DESIL [8] and FLEX [9]. Our mutation arm follows Mut4All's synthesised mutators [3]. IssueMut [10] mines mutators from bug histories, and Fuzz4All [11] generates inputs with a model. AFuzz [12] is the origin of our method, an agent that tests where else a reference bug's cause could live; we add the CIRCT build, validation and triage around it, behind a gate. Delta debugging [13] is stage 5's algorithm. Abstain-and-Validate [14], llvm-autofix [15] and CodeRover-S [16] make a repair agent more selective. Our gate sits downstream and discards by default.

VII. Artefact and Limitations

We release the loop as CHIA nodes and tools that meet at a frozen contract of seven record types, with 736 tests, 686 of which need no cluster, network or model. Four pieces are prepared for CHIA upstream, none merged yet: the assertions-on image, its build workflow, three helpers for its CIRCT module, and the branch that lets its repair chain select our backend. The corpus and every input, verdict, reduced case, transcript and gate decision are at https://github.com/DevAbhinav-23/CHIA_Hackathon_2k26.

Limitations. No maintainer has yet confirmed the bugs, and none is filed. The campaigns reached only 43 of the 187 seeds, so the bugs we found are not exhaustive; throughput was about 3.6 seeds an hour across two shards, bound by Vertex rate limits. The second oracle applies to 6 of the 187 seeds, and the seeds rest on a subject-keyword filter: we read no diff to confirm that each is a bug fix. The contamination screen misses a sibling fixed in a commit whose subject does not name it. The loop finds crashes and assertion failures, and the oracle that would find wrong output only reports.

Acknowledgements and disclosure. We thank the CHIA developers and the A^3 organisers. Claude (Anthropic) helped draft text and code. The authors answer for every claim, number and decision.

VIII. Conclusion

A repair agent for a hardware compiler need not wait for people to report bugs. Reading past fixes exposes sibling defects, restored assertions make them observable, and a gate of tools

discards almost everything else. Fifteen new CIRCT bugs went through that gate where a mutation baseline sent none in the same campaigns. The loop turns the human event from writing a bug report into reading one, and one person decides whether to file it.

REFERENCES

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[2] A. Cui *et al.*, "CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research," *arXiv preprint arXiv:2606.27350*, 2026.

[3] B. Wang *et al.*, "Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports," *arXiv preprint arXiv:2507.19275*, 2025.

[4] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[5] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.

[6] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, "Fuzzing MLIR compiler infrastructure via operation dependency analysis," in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024.

[7] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, "Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation," in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.

[8] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, "DESIL: Detecting silent bugs in MLIR compiler infrastructure," *arXiv preprint arXiv:2504.01379*, 2025.

[9] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, "Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR," in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.

[10] L. Liu, F. Qin, O. Legunsen, and M. d'Amorim, "Learning compiler fuzzing mutators from historical bugs," in *Proc. IEEE/ACM Int. Conf. on Mining Software Repositories (MSR)*, 2026.

[11] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2024.

[12] J. Park and I. Yun, "Agentic fuzzing: Opportunities and challenges," *arXiv preprint arXiv:2605.10074*, 2026.

[13] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[14] J. Cambronero *et al.*, "Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair," in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 118–129.

[15] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, "Agentic harness for real-world compilers," *arXiv preprint arXiv:2603.20075*, 2026.

[16] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, "Fixing security vulnerabilities with agentic AI in OSS-Fuzz," in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026.